Ch13. 인증과 권한 관리

인증: 쿠버네티스 클러스터에 접근할 때 허가 받은 사용자인지 확인

권한 관리: 인증 받은 사용자에게 특정 자원의 사용권한을 부여하는 방법

---

1. 인증

- kube-apiserver는 테스트 목적으로 localhost:8080에서 HTTP 서버를 실행
- 일반적인 HTTPS 인증은 접근하는 클라이언트에게 인증서를 요구하지 않는다.
- 하지만 사용자가 쿠버네티스의 API에 접근하려면 인증을 거쳐야 한다. 외부에서 쿠버네티스의 API에 접근할 수 있는 포트는 **6443**이고 **TLS 인증이 적용**되어 있다.
- 6443 포트에 접근해 통신하려면 kube-apiserver에 있는 인증서와 클라이언트에 있는 인증서 사이의 검증을 통과해야 한다.

쿠버네티스는 일반적인 사용자 계정과 서비스 계정으로 인증을 요청

- 일반적인 사용자 계정은 구글계정이나 오픈스택의 키스톤, LDAP 등 별도의 외부 인증 시스템에 있는 사용자 정보를 연결해 사용. 쿠버네티스는 기본적으로 사용자 정보를 저장하지 않기 때문
- **서비스 계정은 쿠버네티스가 직접 관리하는 계정**. 해당 서비스 계정에는 시크릿이 할당되어 비밀번호 역할을 한다.

1.1 kubectl의 config 파일에 있는 TLS 인증 정보 구조 확인하기

쿠버네티스는 **kube-apiserver와 통신할 때의 기본 인증 방법으로 TLS**를 사용

- **TLS 인증은 통신할 때 오가는 패킷을 암호화**한다.
- ~/.kube/config 파일 내용

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTi ...
    server: https://10.100.0.105:6443
  name: kubernetes
```

```
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: LS0tLS1CRUd ...
```

1.2 서비스 계정 토큰을 이용해 인증하기

kubectl get serviceaccount

- 현재 서비스 계정 확인

```
$ kubectl get serviceaccount
NAME      SECRETS   AGE
default   0         282d
$ kubectl get sa default -o yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: "2024-04-01T01:28:22Z"
  name: default
  namespace: default
  resourceVersion: "296"
  uid: d45bb321-45b7-490b-b42b-a7306b19e1b1
```

kubectl get sa default -o yaml

- **기본 서비스 계정인 default** 확인

---

2. 권한 관리

쿠버네티스에서는 권한 관리에 필요한 여러 가지 방법을 제공한다. 크게 **ABAC(Attribute-Based Access Control)와 RBAC(Role-Based Access Control)** 방법이 있다.

- **ABAC**
  - **속성 기반의 권한 관리**
  - 사용할 수 있는 속성으로는 사용자, 그룹, 요청 경로, 요청 동사 등이 있다.
  - 네임스페이스나 자원 등을 속성으로 사용할 수 있다.
  - 권한 설정 내용을 파일로 관리하므로 권한을 변경하려면 직접 마스트에 접속해서 파일을 변경한 후 kube-apiserver 컴포넌트를 재시작해야 하므로 관리하기 번거롭다.
- **RBAC**
  - **역할 기반 권한 관리**로 대부분의 권한 관리 시스템에서 많이 사용
  - 사용자와 역할을 별개로 선언한 후 2가지를 조합해서 사용자에게 권한을 부여
  - 마스터에 접근할 필요 없이 kubectl이나 API를 이용해서 관리
  - **쿠버네티스는 기본 권한 관리로 RBAC를 설정해 관리**

2.1 롤

롤: 특정 API나 자원 사용 **권한들을 명시해둔 규칙의 집합**

- 일반 롤은 해당 롤이 속한 네임스페이스에만 적용
- 롤 설정 예(rbac/read-role.yaml)

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-clusterrole
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

- .kind: Role
- .metadata.name: 롤의 이름, namespace는 없으므로 default
- .rules[]: 해당 롤이 갖는 권한의 규칙을 구체적으로 설정

- apiGroups: 롤이 사용할 API 그룹들을 설정
- resources: 어떤 자원에 접근할 수 있는지 명시
- verbs: 동사라는 단어 뜻 그대로 어떤 동작을 할 수 있는지 설명

[.rules[].verbs 필드에 설정할 수 있는 값]

.rules[].verbs 필드	설명
Create	새로운 자원 생성
Get	개별 자원 조회
List	여러 개 자원 조회
Update	기존 자원 내용 전체 업데이트
Patch	기존 자원 중 일부 내용 변경
Delete	개별 자원 삭제
DeleteCollection	여러 개 자원 삭제

- 전체 파드가 아니라 특정 파드에만 규칙을 설정할 수도 있다. 이때는 .rules[] 필드 하위에 .resourceNames: ["특정파드이름"]을 추가

```
rules:
- apiGroups" [""]
  resources: ["pods"]
  resourceNames: {"mypods"]
  verbs: ["get", "list"]
```

2.2 클러스터롤

ClusterRole: 특정 네임스페이스 사용 권한이 아닌 **클러스터 전체 사용 권한을 관리**

- 클러스터롤의 설정 예(rbac/read-clusterrole.yaml)

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-clusterrole
rules:
- apiGroups: [""]
```

```
  resources: ["pods"]
  verbs: ["get", "list"]
```

- .metadata 필드가 롤과 다른 점은 하위에 .namespace 필드가 없다는 점
  - 클러스터롤은 전체 네임스페이스에 적용하므로 별도의 네임스페이스를 명시하지 않는 것
- 해당 클러스터롤은 클러스터 내 전체 네임스페이스의 Pods에 “get”, “list”를 설정할 수 있다.
- kubectl apply -f read-clusterrole.yaml
- kubectl describe clusterrole read-clusterrole

2.3 롤바인딩

RoleBinding: **롤과 사용자를 묶는 역할**

- 권한은 롤과 클러스터롤을 구분하는 것처럼 바인딩도 롤바인딩과 클러스터롤바인딩으로 구분
- 롤바인딩은 특정 네임스페이스 하나에 적용하고 클러스터롤바인딩은 한번 설정했을 때 클러스터 전체에 적용된다는 점도 비슷
- 롤에 바인딩할 사용자 설정(rbac/serviceaccount-myuser.yaml)
  - .kind 필드: ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myuser
  namespace: default
```

- kubectl apply -f serviceaccount.yaml

- 롤바인딩 설정 예(rbac/read-rolebinding.yaml)

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-rolebinding
```

```
  namespace: default
subjects:
- kind: ServiceAccount
  name: myuser
  apiGroup: ""
roleRef:
  kind: Role
  name: read-role
  apiGroup: rbac.authorization.k8s.io
```

2.4 클러스터롤바인딩

ClusterRoleBinding: **클러스터롤과 사용자를 묶는다.**

- 클러스터롤바인딩의 설정 예(rbac/read-clusterrolebinding.yaml)

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: read-clusterrolebinding
subjects:
- kind: ServiceAccount
  name: myuser
  namespace: default
  apiGroup: ""
roleRef:
  kind: ClusterRole
  name: read-clusterrole
  apiGroup: rbac.authorization.k8s.io
```

Role 설정

1. 네임스페이스(어디)
   kubectl create ns 네임스페이스명
2. 서비스어카운트(누가)
   kubectl create serviceaccount 계정명 -n 네임스페이스명

3. 롤(무엇을, 어떻게)
   kubectl create role 롤명 -n 네임스페이스명 --resource=리소스 --verb=권한
4. 롤바인딩=서비스어카운트+롤(매핑)
   kubectl create rolebinding 롤바인딩명 --role=롤명 --serviceaccount=네임스페이스명:계정명 -n 네임스페이스명

ClusterRole 설정

1. 서비스어카운트(누가)
   kubectl create serviceaccount 계정명 -n 네임스페이스명
2. 클러스터롤(무엇을, 어떻게)
   kubectl create clusterrole 클러스터롤명 --resource=리소스 --verb=권한
3. 클러스터롤바인딩=서비스어카운트+클러스터롤(매핑)
   kubectl create clusterrolebinding 클러스터롤바인딩명 --role=클러스터롤명 --serviceaccount=네임스페이스명:계정명 -n 네임스페이스명